GeoCrop: Design Artifacts and Design Evolution

F26-155-GeoCrop | Base architecture and implementation, October 2026

This document collects the design artifacts of the platform (architecture, components, interfaces, decisions, use cases, domain model, data design, process and deployment views) and shows how the design evolved from the proposal-stage prototype. The guiding principle is *send only what is visible*: results are stored once as rasters, the browser requests small tiles, and heavy work runs in the background.

1 Design Evolution

The proposal prototype was a Django application with Leaflet. It returned the crop-type result for AOI1 as one GeoJSON response (43.3 MB, 101,435 polygons) that the browser parsed and drew as SVG. Uploads were recorded but never processed. Profiling this showed that it would not scale to more images or models, and the panel asked for a light, performance-focused system. The design was redesigned around rasters and tiles.

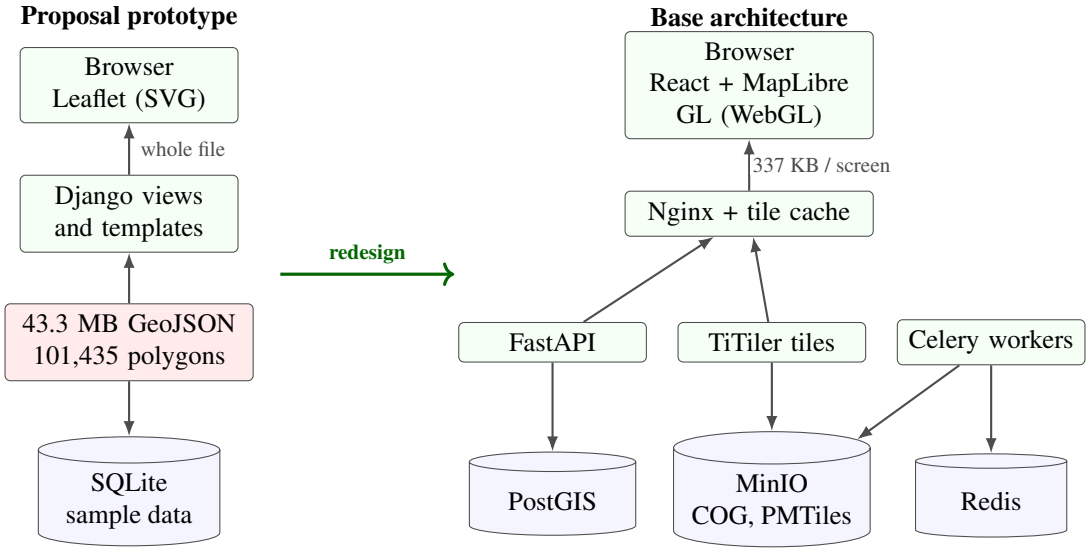


Figure 1: Design evolution: from one large GeoJSON response to tile-based delivery with background processing.

Table 1: Design evolution since the proposal

Aspect	Proposal prototype	Current design	Reason
Result format	GeoJSON polygons (43.3 MB)	COG rasters + PMTiles	Data volume and rendering cost
Map client	Leaflet, SVG	MapLibre GL, WebGL tiles	Smooth zoom with many layers
Backend	Django views and templates	FastAPI + React SPA	Async API, typed contracts, separation of concerns
Database	SQLite	PostgreSQL + PostGIS	Spatial queries, concurrency, catalogue
Processing	None (upload recorded only)	Celery jobs with progress	Real analysis of new imagery
Models	Offline scripts writing GeoJSON	Registry plug-ins writing rasters	Add models without code changes
Dashboard data	Hard-coded sample fields	Pre-computed statistics in PostGIS	Real results
Deployment	Single process	Seven containers via Compose	Reproducible, scalable services

2 Architecture Artifacts

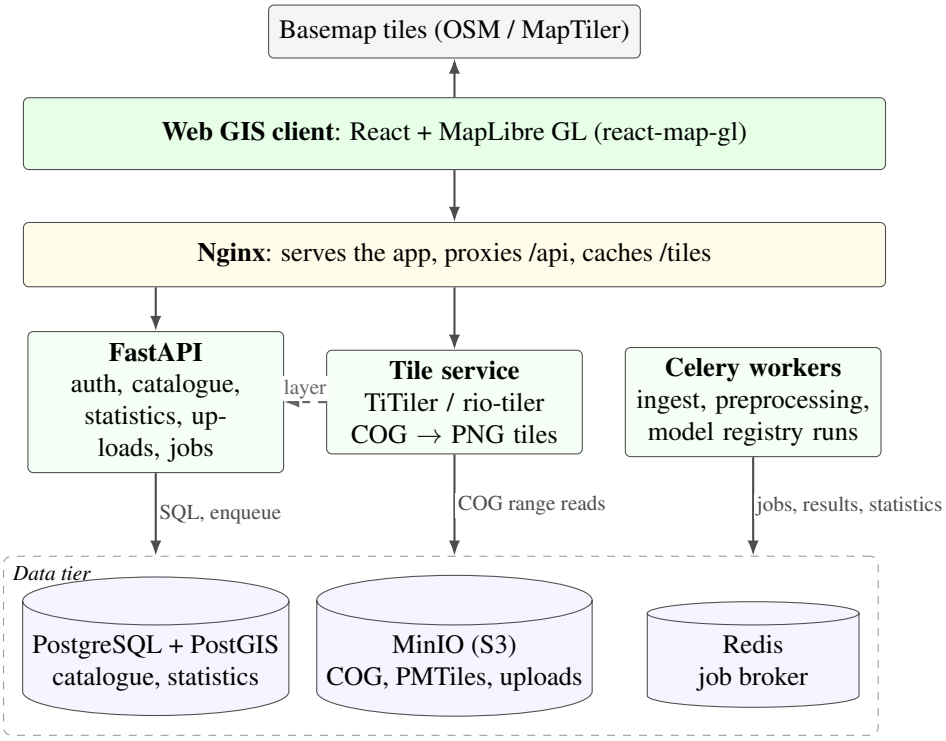


Figure 2: GeoCrop system architecture. The browser also reads PMTiles and uploads imagery directly to object storage using short-lived signed URLs; the tile service looks up layer styles from the API.

Table 2: Architectural components

Component	Technology	Responsibility
Web GIS client	React, Vite, TypeScript, MapLibre GL via react-map-gl, TanStack Query, Zustand	Home, Live Map workspace, Dashboard, Analyze Imagery, Processing, Models, Status, accounts. Builds layers from the catalogue; loads tiles on demand.
Edge	Nginx	Serves the built app; proxies the API; caches tiles with immutable headers; resolves services through Docker DNS at request time.
API service	FastAPI, Pydantic, SQLAlchemy 2, GeoAlchemy2, PyJWT, bcrypt	REST API v1: health, auth, areas, models, layers, fields, point query, statistics, uploads, runs, jobs.
Tile service	TiTiler / rio-tiler (FastAPI)	`/tiles/{layer}/{version}/{z}/{x}/{y}.png`; reads only needed COG blocks; applies the server-side style; optional crop filter.
Workers	Celery	Ingest (validate, reproject, COG), model runs through the registry, statistics, layer publishing, progress.
Model registry	Python package with YAML manifests	Discovers models, resolves dependencies, runs tasks with a common context; built-in cropland, crop type, NDVI index, boundary and yield placeholders.
Spatial database	PostgreSQL 16 + PostGIS 3.4, Alembic	Catalogue of users, areas, uploads, images, models, runs, layers, statistics, field regions, jobs.
Job broker	Redis 7	Celery broker and result backend.
Object storage	MinIO (S3 API)	Uploads, COG imagery and results, PMTiles; anonymous read only for vector tiles.

Table 3: Main REST interfaces (prefix `/api/v1`; interactive documentation at `/api/docs`)

Endpoint	Purpose
`GET/health`	Status of database, broker, storage, tiles and model registry; configuration notes.
`POST/auth/signup`, `/auth/login`, `GET/auth/me`	Accounts and tokens.
`GET/aois`	Areas visible to the user, with bounds.
`GET/models`	Registered models, status, inputs, outputs, accuracy and per-class scores.
`GET/layers?aoi=&kind=&run=`	Layer catalogue: style, legend, bounds, zoom range, tile URL, version.
`GET/fields?aoi=&q=`	Field regions with crop, area and bounding box.
`GET/point?lng=&lat=&layers=`	Values and labels of several layers at one location.
`GET/stats?layer=`	Pre-computed area and share per class.
`POST/uploads`, `/uploads/{id}/complete`	Signed upload URL; start ingestion with models, location and date.
`POST/runs`, `GET/runs`, `GET/jobs/{id}`	Re-run models; run history; job progress and steps.
`GET/tiles/{layer}/{v}/{z}/{x}/{y}.png`	Raster tiles (tile service, cached by Nginx).

Table 4: Key architecture decisions

Decision	Rationale	Alternative rejected
Raster as source of truth (COG)	Models predict pixels; a 30 m raster of AOI1 is about 1.6 MB, the same result as polygons 43.3 MB.	Vectorising all output to GeoJSON
Tiles via TiTiler	Reads only the COG blocks needed per tile; styles applied on the server; no pre-rendering.	Pre-rendered tile folders; single PNG overlays
PMTiles for vectors	One static file read by range requests; no tile server; cacheable.	GeoJSON; dynamic vector tile server
MapLibre GL	WebGL rendering of raster and vector tiles; open source.	Leaflet (SVG/Canvas)
Versioned, immutable tile URLs	Browsers and Nginx can cache for a year; a new run gets a new URL.	Cache invalidation logic
FastAPI	Same stack as TiTiler; async I/O; typed schemas and generated docs.	Django templates (prototype)
PostgreSQL + PostGIS	Spatial types and indexes, joins between catalogue and statistics, JSONB for styles and metrics.	SQLite (prototype)
Celery + Redis	Long jobs off the request path; retries; progress.	Synchronous processing
S3-compatible storage (MinIO)	Large files, range reads, signed uploads, cloud-portable.	Local disk
Manifest-based model registry	Models change over the project; new models without changes elsewhere.	Hard-coded model calls
Docker Compose	One-command, reproducible stack for the team and the panel.	Manual installation

3 Requirements and Domain Artifacts

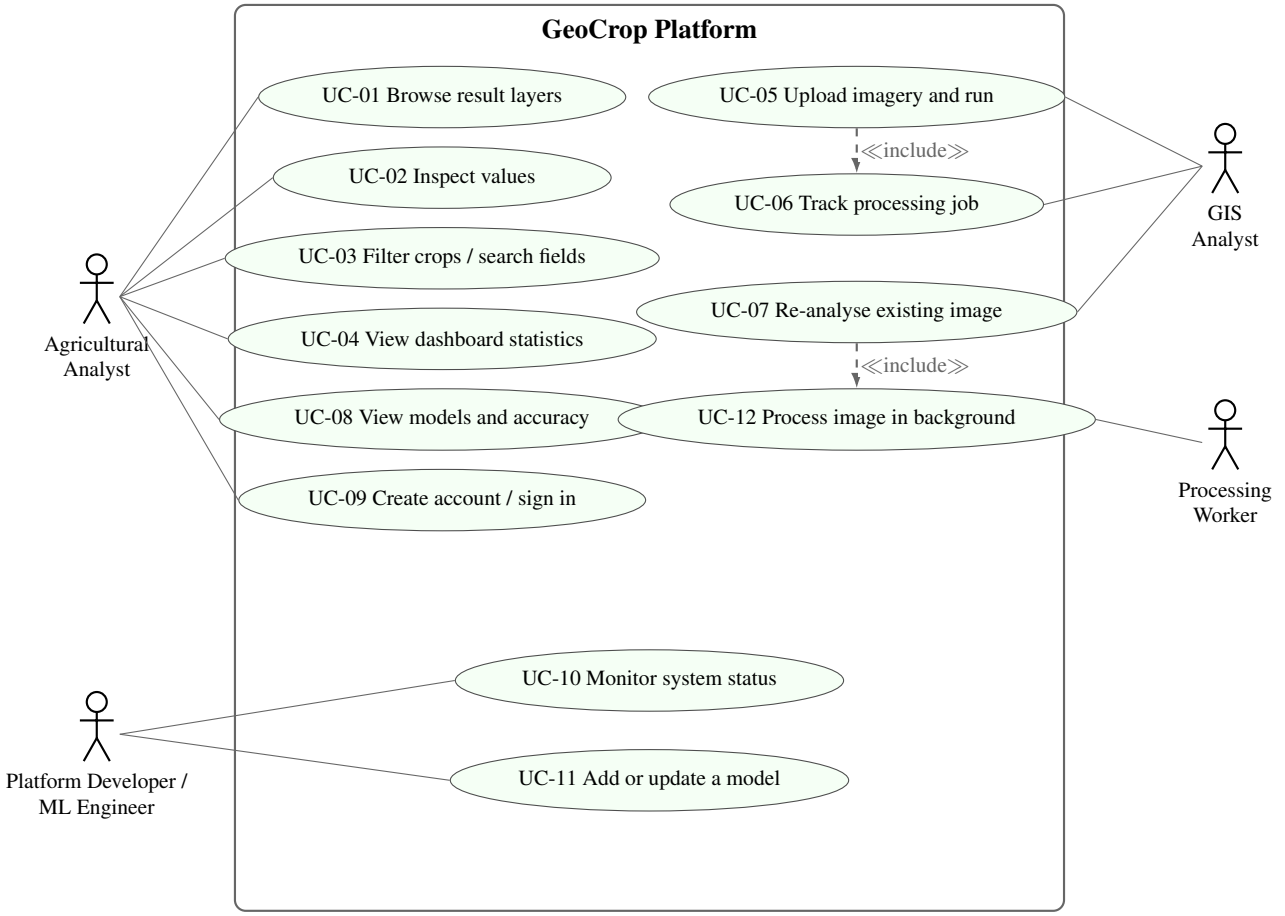


Figure 3: Use case diagram of the GeoCrop platform

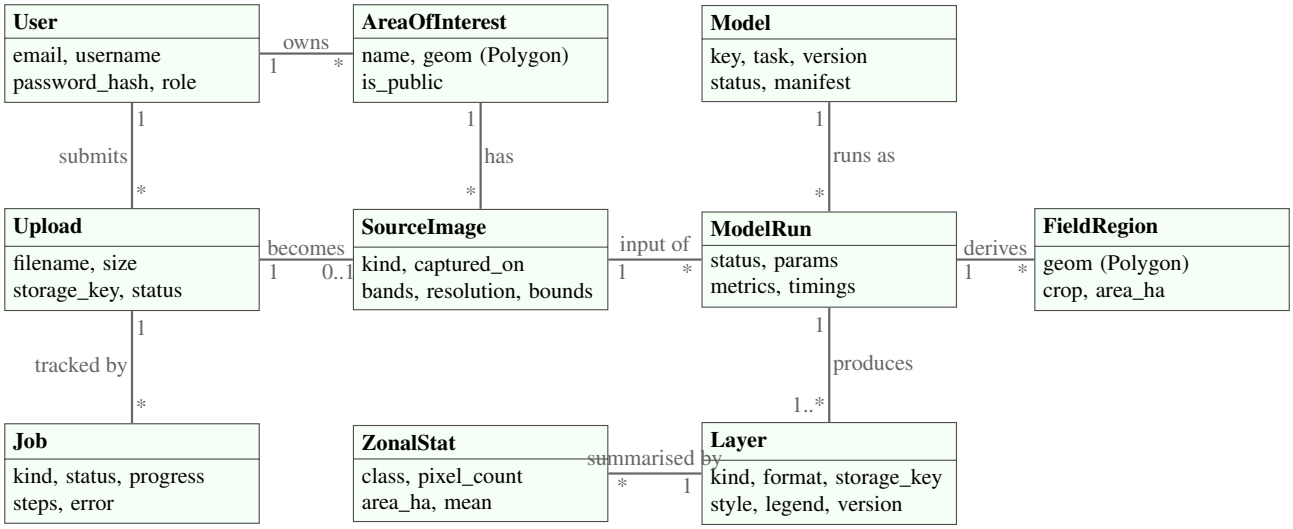


Figure 4: Domain model

4 Data Design

Table 5: Database tables

Table	Key columns
`users`	id, email (unique), username (unique), password_hash, role, created_at
`aoi`	id, owner_id, name, geom (Polygon, GiST), is_public
`upload`	id, user_id, aoi_id, filename, size_bytes, storage_key, status
`source_image`	id, aoi_id, upload_id, kind, captured_on, storage_key, bounds (GiST), band_count, resolution_m
`model`	id, key (unique), task, version, name, status, manifest (JSONB), enabled
`model_run`	id, aoi_id, model_id, source_image_id, status, params, metrics (JSONB), error, timings
`map_layer`	id, run_id, aoi_id, kind, group, name, format, storage_key, bounds (GiST), zoom range, nodata, style, legend (JSONB), version, is_public
`zonal_stat`	layer_id, zone_type, zone_id, class_code, class_name, pixel_count, area_ha, mean, min, max
`field`	id, aoi_id, run_id, geom (Polygon, GiST), attributes (JSONB)
`job`	id, kind, status, progress, message, error, steps (JSONB), upload_id, user_id

5 Behaviour and Deployment Artifacts

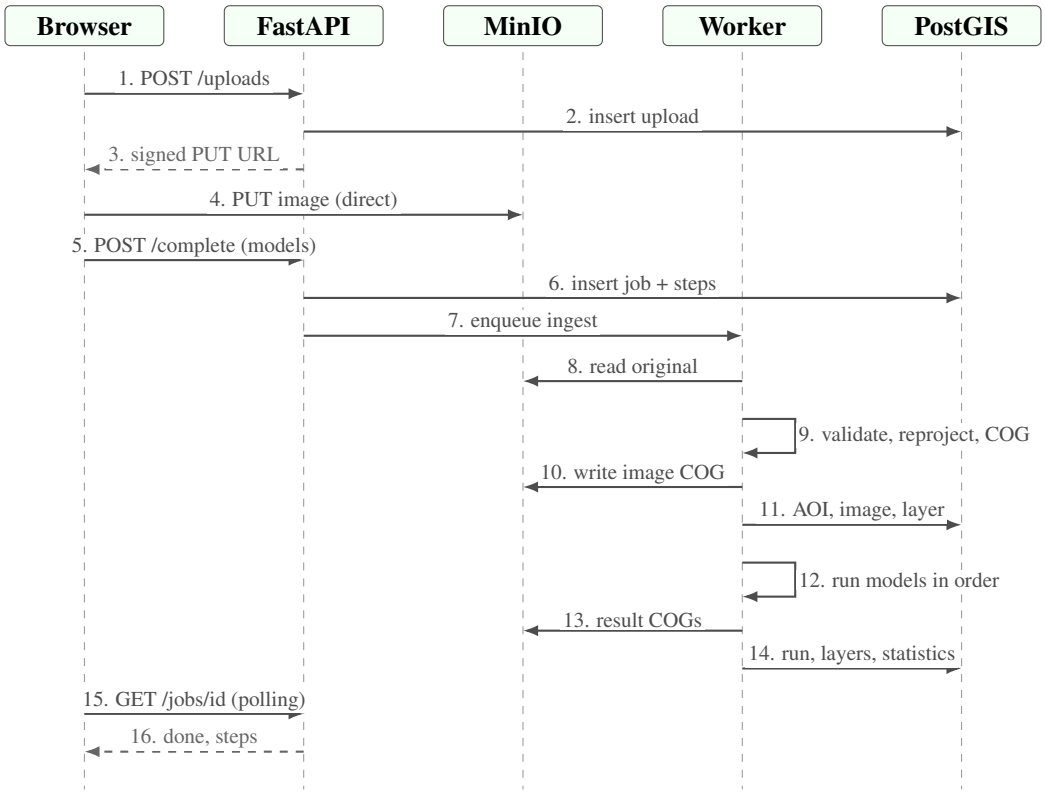


Figure 5: Sequence of an upload and analysis

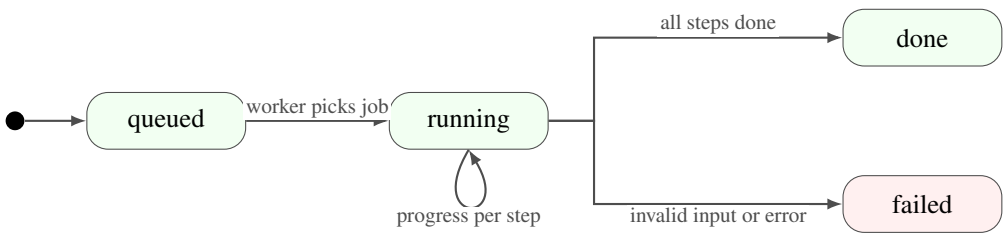


Figure 6: Job state machine

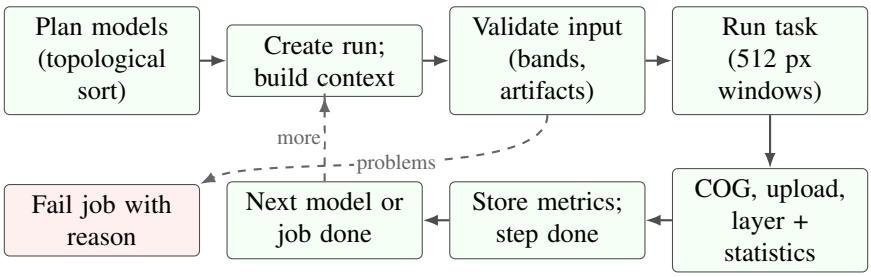


Figure 7: Activity of a model run inside the worker

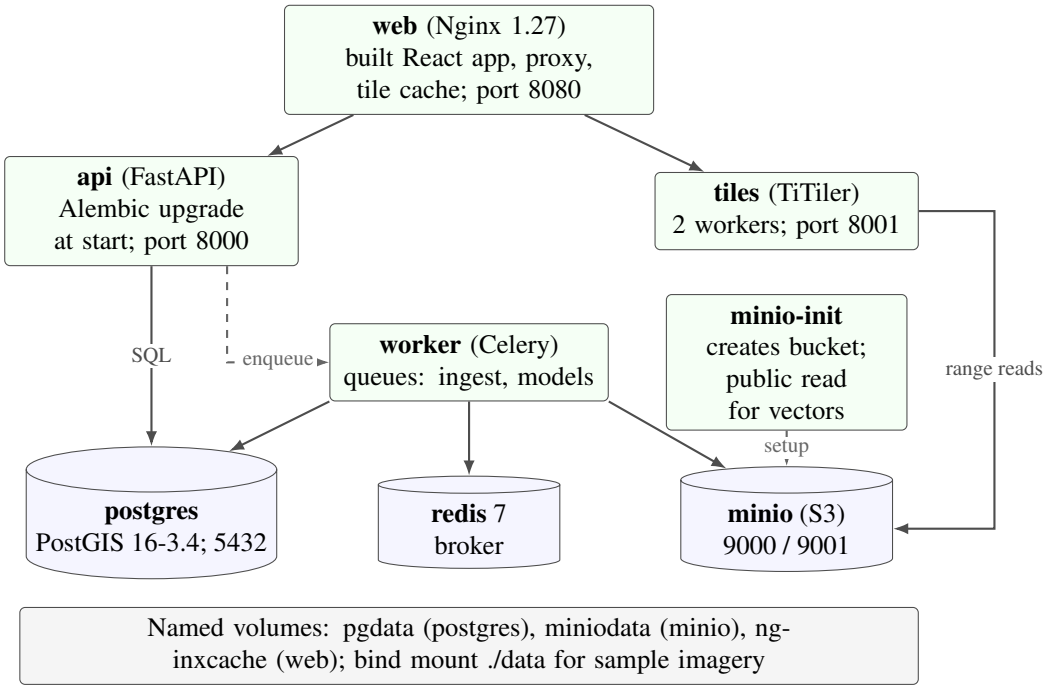


Figure 8: Deployment view of the Docker Compose stack